Introduction

- Recurrent neural networks (RNN) are the state of the art algorithm for sequential data.
- It is the first algorithm that remembers its input, due to an internal memory, which makes it perfectly suited for machine learning problems that involve sequential data.
- In traditional neural networks, all the inputs and outputs are independent of each other, but in cases like when it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words.
- Thus RNN came into existence, which solved this issue with the help of a Hidden Layer.
- The main and most important feature of RNN is **Hidden state**, which remembers some information about a sequence. This state is also known as **memory state.**
- This is why they're the preferred algorithm for sequential data like time series, speech, text, financial data, audio, video, weather and much more

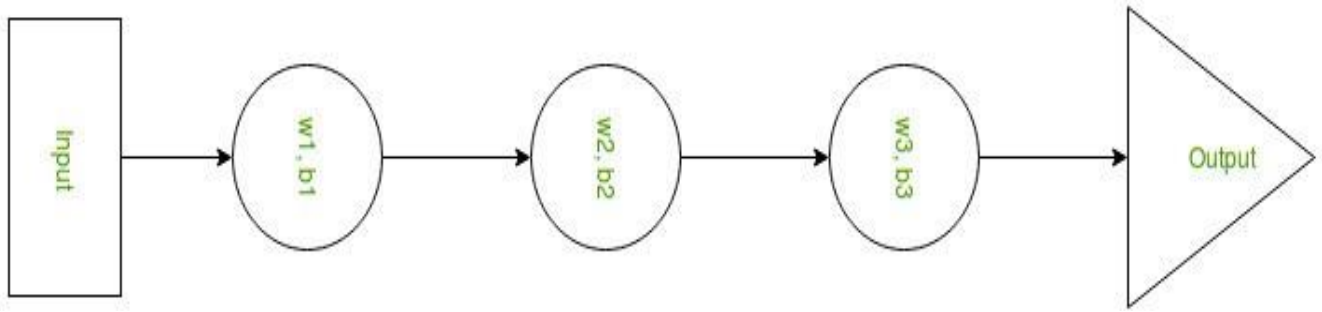
How RNN works

Example:

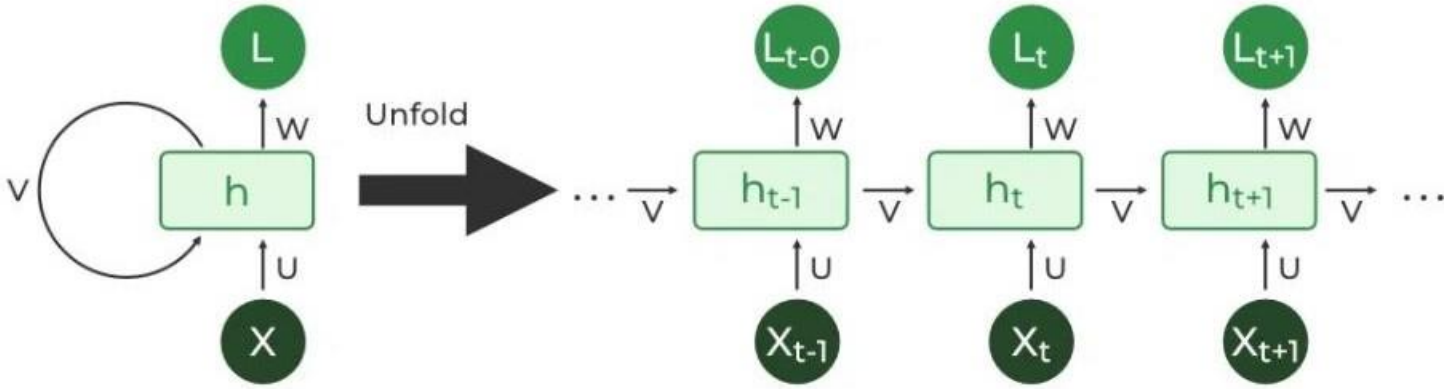
Suppose there is a deeper network with one input layer, three hidden layers and one output layer.

Then like other neural networks, each hidden layer will have its own set of weights and biases, let's say, for hidden layer 1 the weights and biases are (w1, b1), (w2, b2) for second hidden layer and (w3, b3) for third hidden layer.

This means that each of these layers are independent of each other, i.e. they do not memorize the previous outputs.



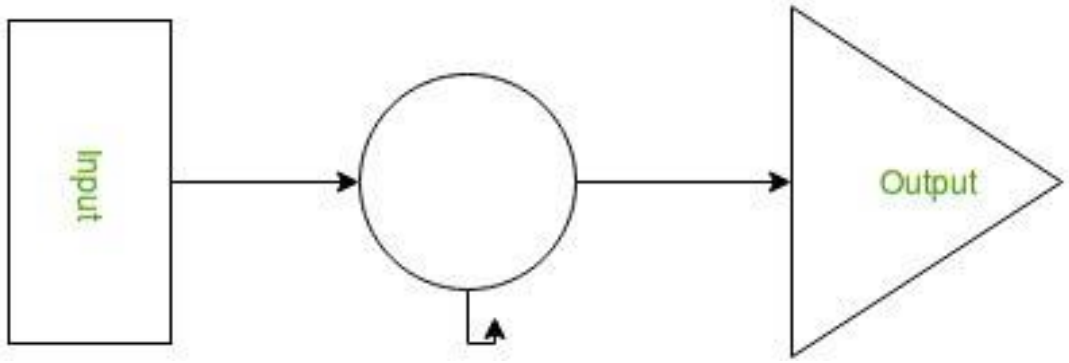
Architecture Of Recurrent Neural Network



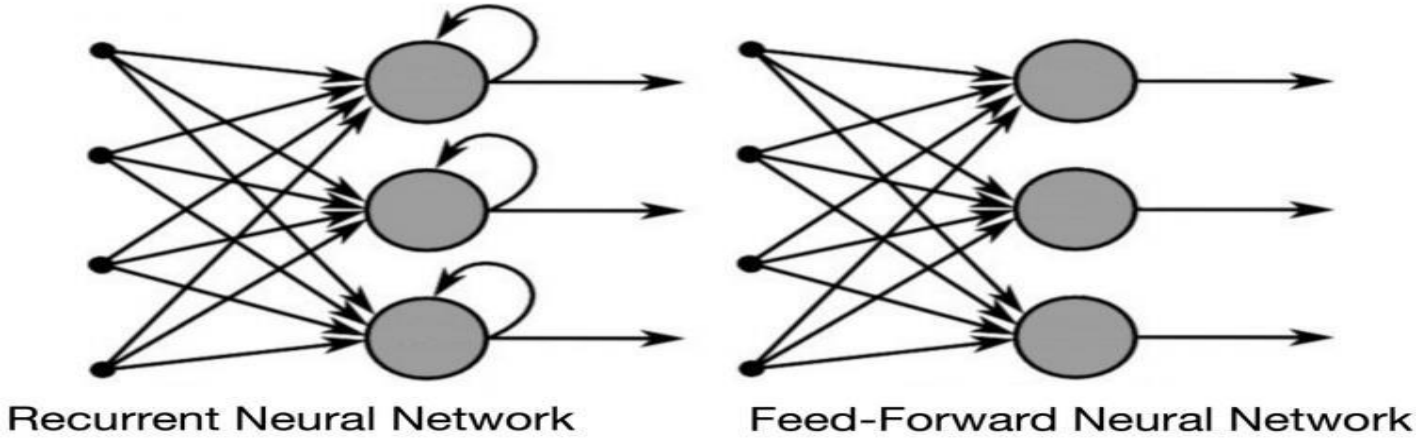
RNNs have the same input and output architecture as any other deep neural architecture. However, differences arise in the way information flows from input to output. Unlike Deep neural networks where we have different weight matrices for each Dense network in RNN, the weight across the network remains the same.

RNN will do the following:

- RNN converts the independent activations into dependent activations by providing the same weights and biases to all the layers, thus reducing the complexity of increasing parameters and memorizing each previous outputs by giving each output as input to the next hidden layer.
- Hence these three layers can be joined together such that the weights and bias of all the hidden layers is the same, into a single recurrent layer



RNN VS. FEED-FORWARD NEURAL NETWORKS



a RNN has two inputs: the present and the recent past.

Backpropagation in RNN

What happens in neural network?

Backpropagation is nothing but going backwards through your neural network to find the partial derivatives of the error with respect to the weights.

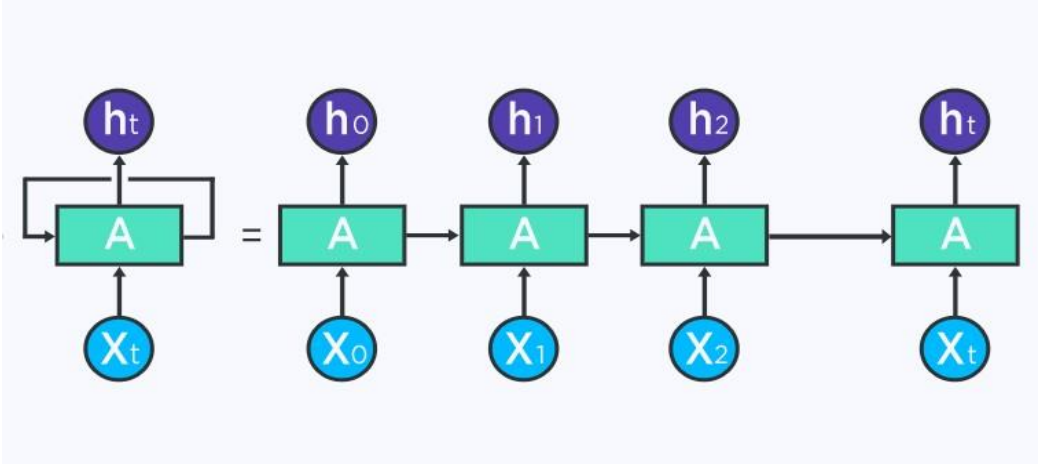
Those derivatives are then used by gradient descent, an algorithm that can iteratively minimize a given function. Then it adjusts the weights up or down, depending on which decreases the error. That is exactly how a neural network learns during the training process.

So, with backpropagation you basically try to tweak the weights of your model while training.

In RNN the neural network is in an **ordered fashion** and since in the ordered network each variable is computed one at a time in a specified order like first h0, h1 then h2 then h3 so on. Hence we will apply backpropagation throughout all these hidden time states sequentially.

Backpropagation in RNN – through time (BPTT)

When we apply a Backpropagation algorithm to a Recurrent Neural Network with time series data as its input, we call it backpropagation through time. A single input is sent into the network at a time in a normal RNN, and a single output is obtained. One timestep will consist of multiple time series data points entering the RNN at the same time.



Formula for calculating the output of RNN

$$h_t = \tanh (W_{hh}h_{t-1} + W_{xh}x_t)$$

W_{hh} -> weight at recurrent neuron

W_{xh} -> weight at input neuron

$$y_t = W_{hy}h_t$$

Y_t -> output

W_{hy} -> weight at output layer

a RNN can be seen as a sequence of neural networks.

During backpropagation the error is backpropagated from the last to the first timestep. This allows calculating the error for each timestep, which allows updating the weights.

Note that BPTT can be computationally expensive when you have a high number of timesteps.

Training through RNN

1. A single time step of the input is provided to the network.
2. Then calculate its current state using set of current input and the previous state.
3. The current h_t becomes h_{t-1} for the next time step.
4. One can go as many time steps according to the problem and join the information from all the previous states.
5. Once all the time steps are completed the final current state is used to calculate the output.
6. The output is then compared to the actual output i.e the target output and the error is generated.
7. The error is then back-propagated to the network to update the weights and hence the network (RNN) is trained.

Advantages

1.An RNN remembers each and every piece of information through time. It is useful in time series prediction only because of the feature to remember previous inputs as well. This is called Long Short Term Memory.

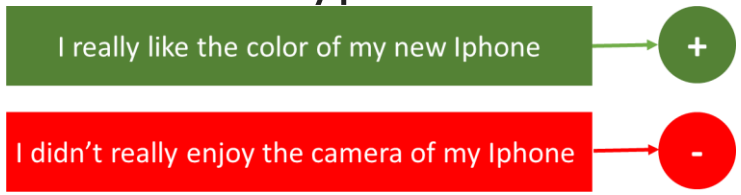
2.Recurrent neural networks are even used with convolutional layers to extend the effective pixel neighborhood.

Disadvantages:

1. Gradient vanishing and exploding problems.
2. Training an RNN is a very difficult task.
3. It cannot process very long sequences

Applications of RNN

1. **Sentiment Classification(many to one)**– This can be a task of simply classifying tweets into positive and negative sentiment. So here the input would be a tweet of varying lengths, while output is of a fixed type and size.



1. **Image Captioning(one to many)**– Here, let's say we have an image for which we need a textual description. So we have a single input – the image, and a series or sequence of words as output. Here the image might be of a fixed size, but the output is a description of varying lengths



Application

3. **Language Translation(many to many)** – This basically means that we have some text in a particular language let's say English, and we wish to translate it in French

